# SYSTEM DESIGN SPECIFICATION(SDS) FOR KCA CONNECT AGENTIC AI

IGNATIUS LUMOSI
22/05647
BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY (BIT 4104)
SUPERVISOR: GRIFFIN KENGA

DATE SUBMITTED: 13/11/2025

# Table of Contents


1. Introduction ..... 1
   - Purpose ..... 1
   - Scope ..... 1
   - Overview ..... 2
   - Definitions, Acronyms, and Abbreviations ..... 2
2. System Overview ..... 3
   - System Context ..... 3
   - Assumptions and Dependencies ..... 3
3. System Architecture ..... 4
4. Database design ..... 5
   - Data Model/Schema ..... 5
   - Data Flow Diagrams ..... 5
   - Data Storage and Security ..... 6
5. Program design ..... 7
6. Interface Design ..... 8
7. Deployment Design ..... 9
8. Security Design ..... 10
   - Audit and Logging ..... 10

References ..... 11

# 1. Introduction

The System Design Specification (SDS) for KCA Connect Agentic AI provides a comprehensive technical blueprint detailing the system's architecture, components, interfaces, and implementation strategy. It translates the Software Requirements Specification (SRS) into a concrete design framework, defining how system components interact and function collectively to achieve institutional and user objectives.

This document aims to ensure that I, the developer, administrators, and stakeholders share a unified understanding of the system's design, guiding its development, deployment, testing, and maintenance phases.

## Purpose

The purpose of this document is to outline the detailed system design and architectural model for the KCA Connect Agentic AI, an intelligent digital assistant built for KCA University. It serves as a bridge between the high-level requirements defined in the SRS and the actual implementation plan.

Specifically, this SDS:

- ✓ Defines system components and their interrelationships.
- ✓ Details data structures, process logic, and information flow.
- ✓ Establishes security and reliability mechanisms.
- ✓ Provides a technical reference for developers, testers, and maintainers.

By offering this level of technical specification, the SDS ensures that all aspects of the system's design align with KCA University's digital transformation goals and user experience standards.

## Scope

The scope of this SDS covers all components necessary to design, develop, and deploy the KCA Connect Agentic AI platform. The system will serve as a web-based communication assistant that integrates with university databases, providing real-time access to academic and administrative information.

This document includes:

- ✓ Frontend and backend system design
- ✓ Database schema and data models
- ✓ System and data flow diagrams
- ✓ Security and privacy mechanisms
- ✓ Deployment and maintenance strategies

## Overview

KCA Connect Agentic AI is an intelligent conversational system designed to enhance service delivery between students, lecturers, and administrative staff at KCA University. It uses natural language understanding and AI-driven reasoning to respond to user queries related to academics, timetables, examinations, announcements, and fees.

The system leverages a cloud-based, modular design and integrates seamlessly with existing university systems via APIs.

## Definitions, Acronyms, and Abbreviations

| Term | Definition |
|---|---|
| **AI** | Artificial Intelligence |
| **NLP** | Natural Language Processing |
| **LLM** | Large Language Model |
| **API** | Application Programming Interface |
| **UI** | User Interface |
| **SRS** | Software Requirements Specification |
| **SDS** | System Design Specification |
| **Qdrant** | Vector database for semantic embeddings |
| **LangChain** | Framework for building context-aware AI applications |

# 2. System Overview

The system will operates as a multi-layered, web-based AI assistant capable of understanding user intent and retrieving contextual responses. It is designed to support real-time interaction, secure data exchange, and scalable performance for the KCA University ecosystem.

Core objectives include:

- ✓ Automating access to academic and administrative information.
- ✓ Providing 24/7 assistance through conversational interfaces.
- ✓ Reducing human workload and response times.
- ✓ Enhancing accessibility for students and faculty through an intuitive interface.

## System Context

KCA Connect Agentic AI will exist within KCA University's digital infrastructure and interact with several internal systems via APIs. It will be hosted on Firebase and integrated with Qdrant for semantic search and a Python-based backend powered by LangChain and the Llama LLM.

interactions will include:

- ✓ User interactions via web browsers.
- ✓ Backend communication with LLM and database.
- ✓ Integration with university systems (timetables, fees, announcements).

The system's environment will ensure data privacy, minimal latency, and smooth integration with KCA University's existing authentication and academic data services.

## Assumptions and Dependencies

The system design assumes:

Stable and reliable internet connectivity (≥10 Mbps).

Active API access to KCA University databases.

Hosting via Firebase or equivalent university servers.

Access to open-source or licensed LLM models (e.g., Llama).

Familiarity of end-users with web browsers and basic digital literacy.

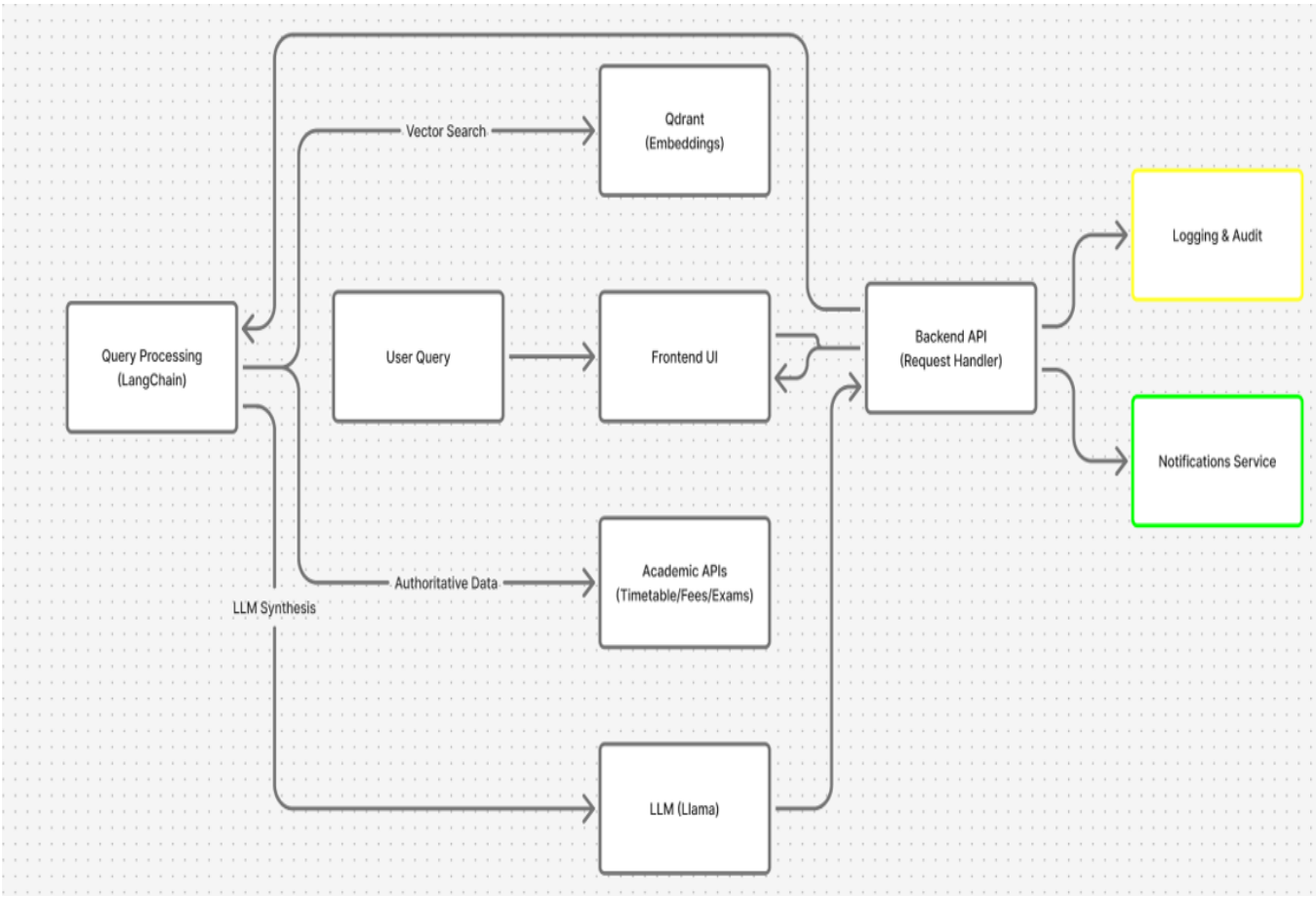
Dependencies include Firebase Authentication, Qdrant, FastAPI, LangChain, and containerization tools (Docker).

# 3. System Architecture

The architecture of KCA Connect Agentic AI follows a modular, service-oriented design enabling scalability, maintainability, and high performance. It comprises three major layers: Presentation (Frontend), Logic (Backend AI Engine), and Data Layer.

1. Frontend Layer: Developed using React and Tailwind CSS, providing a responsive and interactive web interface.
2. Backend Layer: Python FastAPI application integrated with LangChain and Llama LLM for query interpretation and response generation.
3. Data Layer: Firebase for authentication and user management; Qdrant vector database for semantic search; optional integration with academic APIs for timetables, fees, and exams.

The architecture supports RESTful API communication between frontend and backend, ensuring efficient data exchange and system extensibility. Docker containers and microservices design patterns are used for deployment flexibility.



# 4. Database design

## Data Model/Schema

The system uses two primary data stores: Firebase and Qdrant.
- Firebase stores user authentication credentials, roles, and preferences.
- Qdrant stores semantic embeddings for documents such as timetables, fee structures, and FAQs.

## Data Flow Diagrams

Data flows from the user interface to the backend via secure HTTPS requests.

User submits a query via the frontend.

The query is transmitted via HTTPS to the backend.

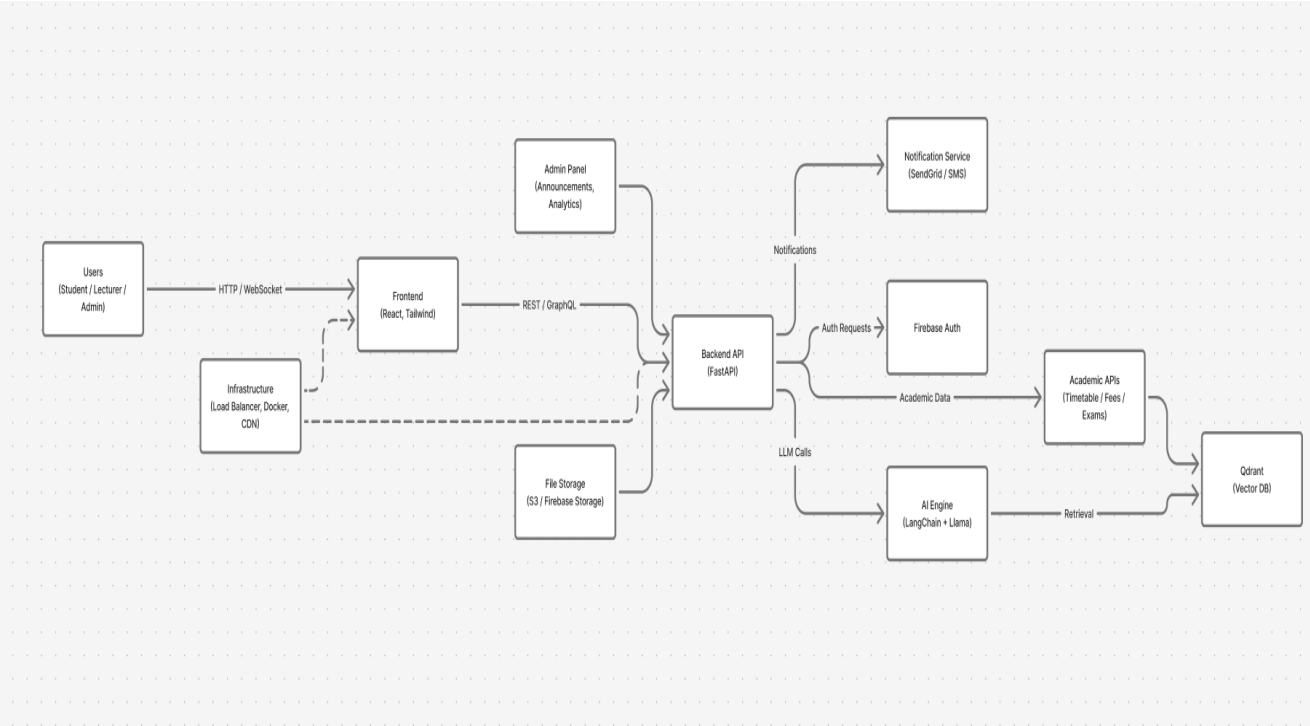
The AI engine interprets the intent and retrieves relevant embeddings.

Contextual data is fetched from APIs or databases.

A synthesized response is returned to the frontend.

This bi-directional flow enables real-time interaction and iterative learning.

.



## Data Storage and Security

All sensitive data is encrypted at rest and in transit.

AES-256 encryption protects stored data in Firebase and Qdrant.

TLS 1.3 secures data during transmission.

Firebase Authentication enforces access control and identity verification.

# 5. Program design

The system consists of independent modules mapped to functional requirements:

- ✓ Authentication Module Manages registration, login, and role-based access control via Firebase.
- ✓ Campus Preference Module Filters data based on user campus and academic context.
- ✓ Query Processing Module Uses LangChain to interpret natural language input and retrieve information from Qdrant or APIs.
- ✓ Timetable Module Displays class timetables customized per student.
- ✓ Fees Module Shows fee balances, payment deadlines, and account details.
- ✓ Announcements Module Pushes notifications and institutional announcements.
- ✓ Exam Module Manages exam schedules, venues, and regulations.
- ✓ Admin Module Provides tools for administrators to manage content and view analytics.

Each module interacts through RESTful endpoints, ensuring modularity, maintainability, and testability.

# 6. Interface Design

The UI follows minimalist design principles to promote ease of navigation and reduce cognitive load.

Key interfaces include:

Login & Registration Screens: Secure forms with validation and password recovery.

Main Dashboard: Integrates chatbot, quick action buttons, and personalized notifications.

Timetable & Fee Pages: Include filters, export options (CSV/PDF), and search features.

Admin Dashboard: Provides analytical insights, post management, and user activity logs.

# 7. Deployment Design

The deployment strategy uses containerized environments (Docker) and supports cloud deployment via Firebase Hosting
or Google Cloud. The architecture includes load balancing, redundancy, and automated backups.

Deployment Phases:
1. Development – Local testing on developer machines.
2. Staging – Integration testing and quality assurance.
3. Production – Deployed on cloud infrastructure accessible to users.
Rollback procedures are implemented using version control and backup restoration.

# 8. Security Design

Security is built into all layers:

Authentication & Authorization: Managed via Firebase, with optional multi-factor authentication for administrators.

Data Encryption: AES-256 and TLS 1.3 ensure data integrity.

Role-Based Access Control (RBAC): Restricts access to system resources.

API Security: All endpoints are protected by HTTPS and token-based access.

Rate Limiting: Prevents brute-force and denial-of-service attacks.

Regular penetration testing and vulnerability scans are conducted as part of maintenance.

## Audit and Logging

System logs all critical activities including logins, API calls, and administrative changes. Logs are stored securely and periodically reviewed for anomalies. Compliance follows Kenya's Data Protection Act (2019) and GDPR where applicable.

# References

- KCA University IT Policy Guidelines (2025)
- LangChain Documentation. https://www.langchain.com
- Qdrant Documentation. https://qdrant.tech
- Firebase Authentication API Reference. https://firebase.google.com
- Kenya Data Protection Act, 2019